Chapter 6
Transactions

Course Notes

CSE0469 Blockchain: Special Topics in SW Engineering

Yusuf Altunel

Based on Mastering Bitcoin 2e; Andreas M. Antonopoulos; O'Reilly, 2017

Content


Introduction


Transactions in Detail


Transaction Outputs & Inputs


Transaction Script & Script Language


Digital Signatures


Bitcoin Addresses, Balances, & Other Abstractions

Bitcoin is Designed for Transactions



Transactions

the most important part

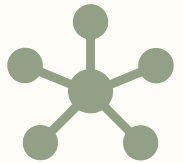
of the bitcoin system.



Most critical part

Everything else in bitcoin

is designed for transactions



Life of Transaction

created

propagated on the network

validated

added to the global ledger

Transactions are Data Structures



data structures

encode the transfer of value

between bitcoin participants



Public Entries

Each transaction is

a public entry in blockchain



Bitcoin Blockchain

the global double-entry

bookkeeping ledger

Spending Concept of Blockchain

UTXO

Unspent Transaction Output

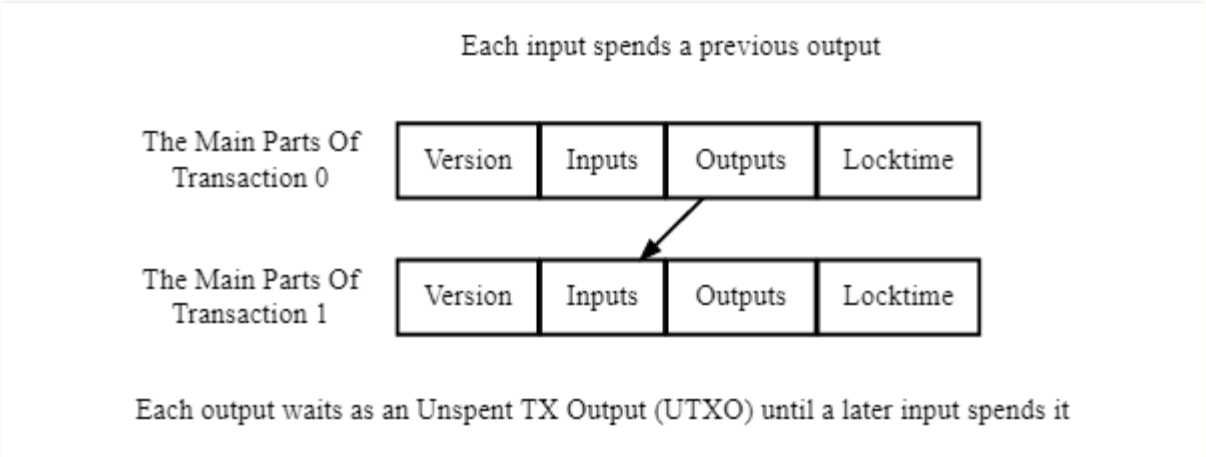
TXO

Transaction Output

Spending

TXO is unspent

Until appears in Input of a Transaction



Chapter 6

Examine
various forms of transactions

Containment
what they contain

Creation
how to create them

Verification
how they are verified



Stone
how they become permanent records



Transactions in Detail

Transactions: Behind the Scenes

Example Alice’s Case



The block explorer

shows a transaction



Adresses

from Alice’s “address”

to Bob’s “address.”



a simplified view

what is contained

in a transaction



Artificial contstruction

much of the information shown

is constructed by the **block explorer**

Transaction View information about a bitcoin transaction

0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK (0.1 BTC - Output)



1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA - (Unspent) 0.015 BTC

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK - (Unspent) 0.0845 BTC

97 Confirmations | 0.0995 BTC

Summary	
Size	258 (bytes)
Received Time	2013-12-27 23:03:05
Included In Blocks	277316 (2013-12-27 23:11:54 +9 minutes)

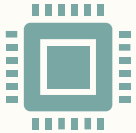
Inputs and Outputs	
Total Input	0.1 BTC
Total Output	0.0995 BTC
Fees	0.0005 BTC
Estimated BTC Transacted	0.015 BTC

Table 4-1. Base58Check version prefix and encoded result examples

Type	Version prefix (hex)	Base58 result prefix
Bitcoin Address	0x00	1
Pay-to-Script-Hash Address	0x05	3
Bitcoin Testnet Address	0x6F	m or n

Table 4-1 first appeared on Chapter 4: Keys & Addresses as slide 71

Transactions—Behind the Scenes



Artificial constructs

represented information

not in the bitcoin system



Bitcoin command line

Bitcoin Core's command-line interface



Commands

getrawtransaction

decoderrawtransaction



Alice's "raw" transaction

retrieve

decode

see what it contains

```
{
        "version": 1,
        "locktime": 0,
        "vin": [
                {
                        "txid":
"7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18",
                        "vout": 0,
                        "scriptSig" :
"3045022100884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb02204
b9f039ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e3813[ALL]
0484ecc0d46f1918b30928fa0e4ed99f16a0fb4fde0735e7ade8416ab9fe423cc5412336376789d1
72787ec3457eee41c04f4938de5cc17b4a10fa336a8d752adf",
                        "sequence": 4294967295
                }
        ],
        "vout": [
                {
                        "value": 0.01500000,
                        "scriptPubKey": "OP_DUP OP_HASH160
ab68025513c3dbd2f7b92a94e0581f5d50f654e7 OP_EQUALVERIFY OP_CHECKSIG"
                },
                {
                        "value": 0.08450000,
                        "scriptPubKey": "OP_DUP OP_HASH160
7f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a8 OP_EQUALVERIFY OP_CHECKSIG",
                }
        ]
}
```

Transaction Content



Where is

Alice's address?

Bob's address?

the 0.1 input "sent" by Alice?



Not part of bitcoin

coins

senders

recipients

balances

accounts

addresses

Transaction Content



Constructed

to make things easier

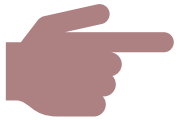
to understand



Real transactions

made up of actual data

different from what users see



Seems to contain

strange & indecipherable fields

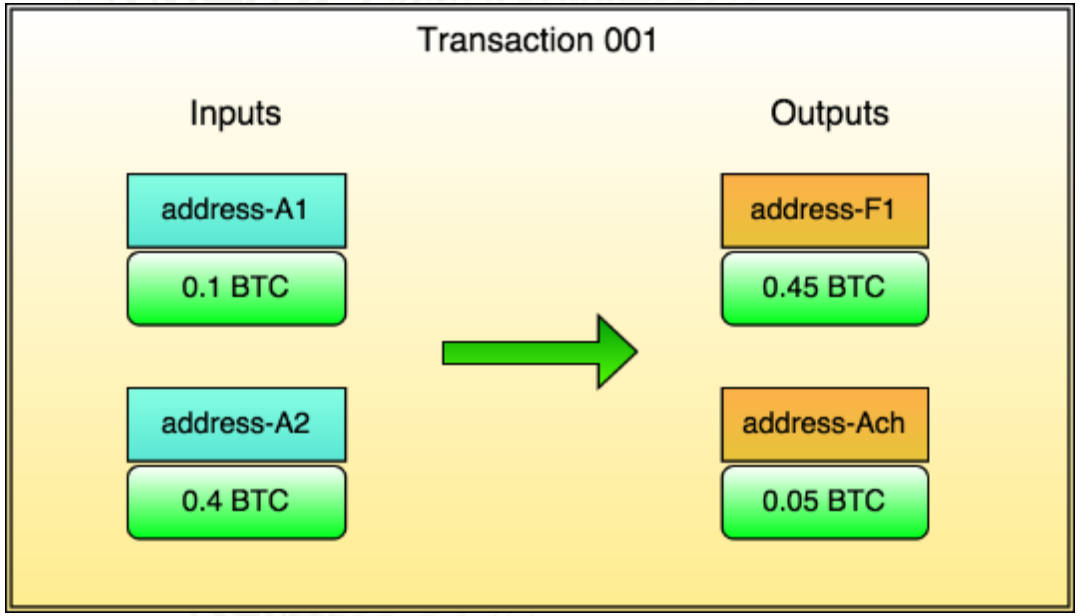
hexadecimal strings

Transaction Outputs & Inputs

TXO &TXI

- Transaction Outputs
- Transaction Inputs
- Transaction Fees
- Adding Fees to Transactions

Transaction Outputs & Inputs



transaction output

The fundamental building block

of a bitcoin transaction

Indivisible Chunks

Transaction outputs

are indivisible chunks of bitcoin currency

Resides on

recorded on the blockchain

Transaction Outputs & Inputs

Validity

recognized as valid

by the entire network

UTXO

known as

unspent transaction outputs

Spendable

all available

and **spendable outputs**

UTXO Set



UTXO set

The collection of all UTXO



millions of UTXO's

at any time

grows & shrinks



Grows

when new UTXO

is created



Shrinks

when UTXO is consumed

means spent



Transactions

changes the UTXO set

and its state

Wallet Balance

Receiving Bitcoin

Wallet detects UTXO

Can be spent with a key

Wallet's key

Transaction output

contains a bitcoin address

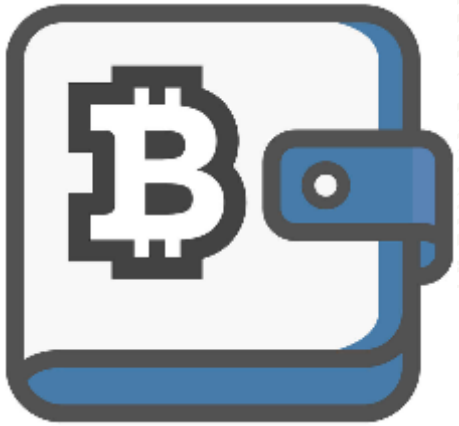
Bitcoin Address

Generated from public key

Balance

is the **sum of all UTXO**

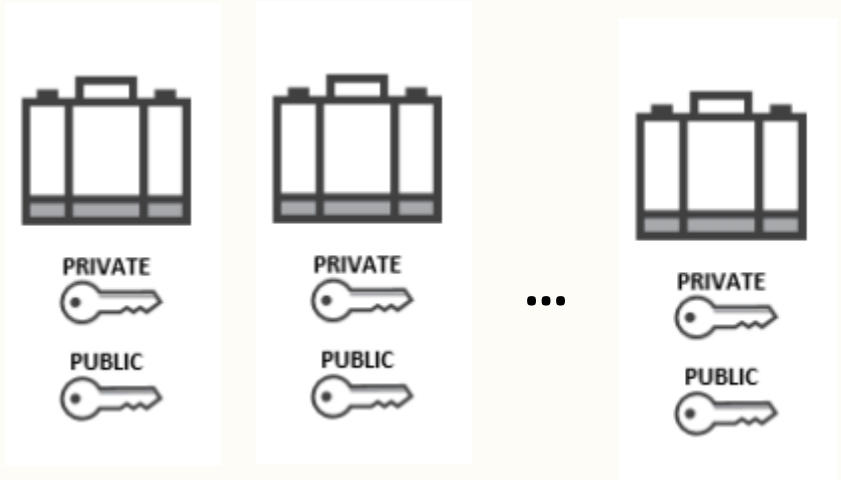
user's wallet can spend





Wallet Balance







UTXO Scattered

among hundreds of transactions

hundreds of blocks

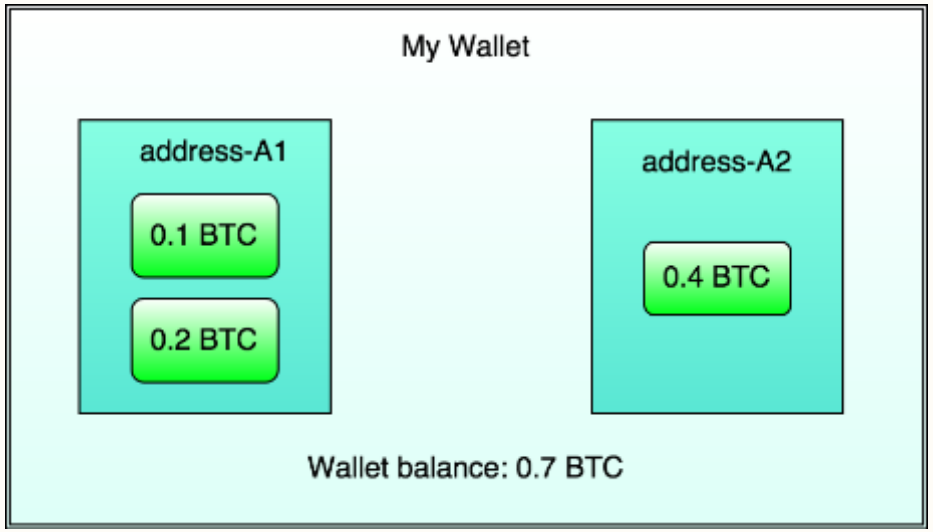


calculation

scanning the blockchain

aggregating the value of UTXO

Wallet Balance



balance is a concept

created by

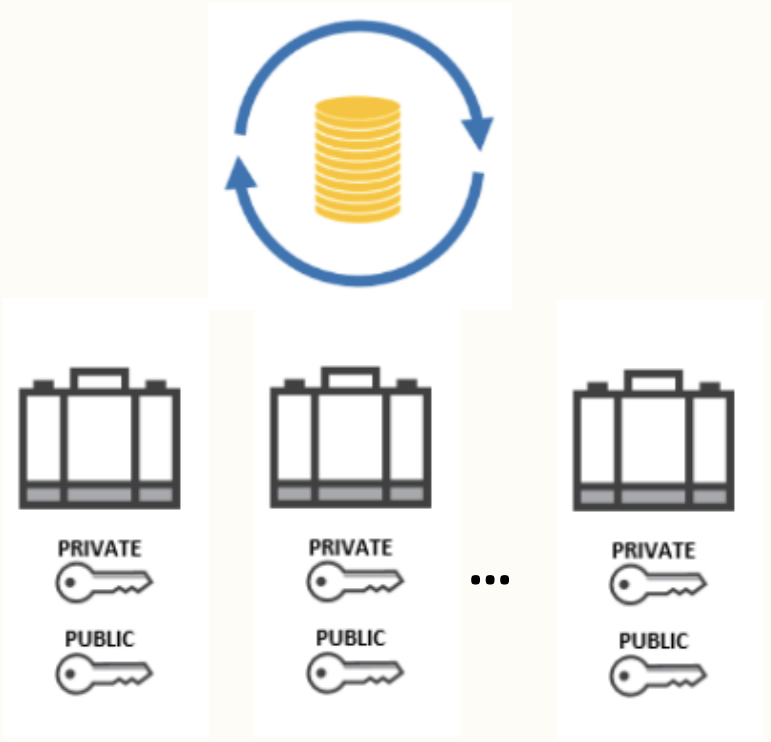
the **wallet application**

Condition To Spend

the wallet can spend UTXO

with the keys it controls

Wallet Database



wallets

maintain a database

or use a database service

Reference

store a quick reference set

of all the UTXO

Quick access

to the UTXO

they can spend

with the keys they control.

Transaction Value Bitcoin vs Satoshi

fx

A transaction output

can have

an arbitrary integer value

Value

is denominated

as a multiple of **Satoshies**

0.0

Division of Bitcoin

bitcoin can be divided down

to Satoshies



Bitcoin vs Satoshi

1BTC = 100 000 000 Satoshies

Indivisible Units of Value



Output

can have any arbitrary value



Indivisible

once created

value is **indivisible**



Discrete

outputs are

discrete units of values



consumption

An unspent output can

consumed in its **entirety**

Change





UTXO larger?

If an UTXO is larger

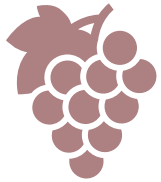
than the desired value



Entirety consumed

it must still be consumed

in its entirety

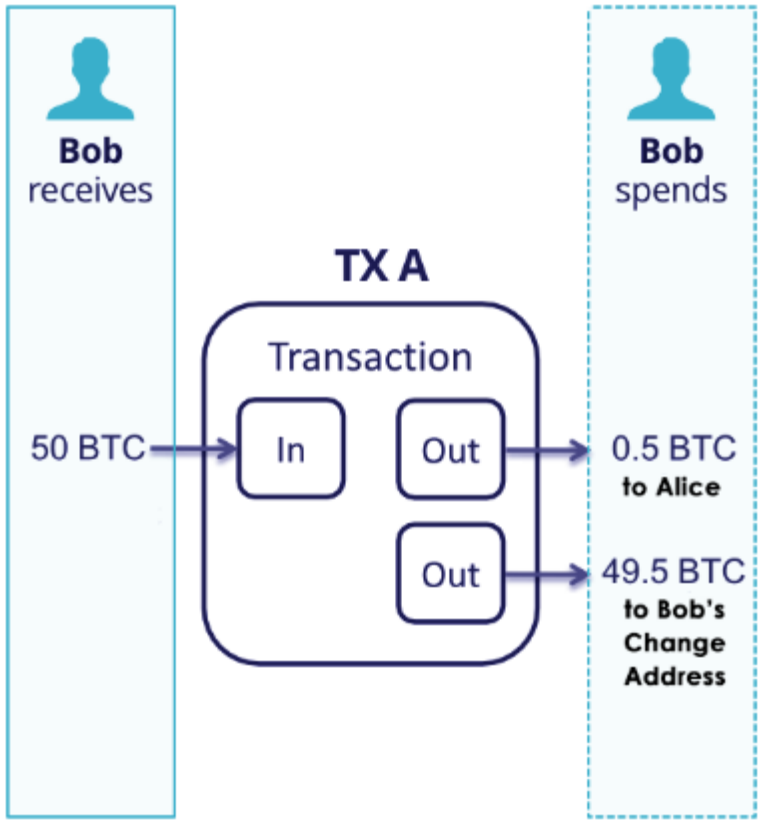


Indivisible nature

As a result of

the indivisible nature

Change



Example

UTXO worth 5**0** bitcoin

to pay only 0.5 bitcoin

First UTXO

paying **0,5** bitcoin

to recipient

Second UTXO

paying **49.5** bitcoin

back to own wallet

Move of Value

transaction creates

new transaction outputs

spend in future

new UTXO can be consumed

by a future transaction

value move forward

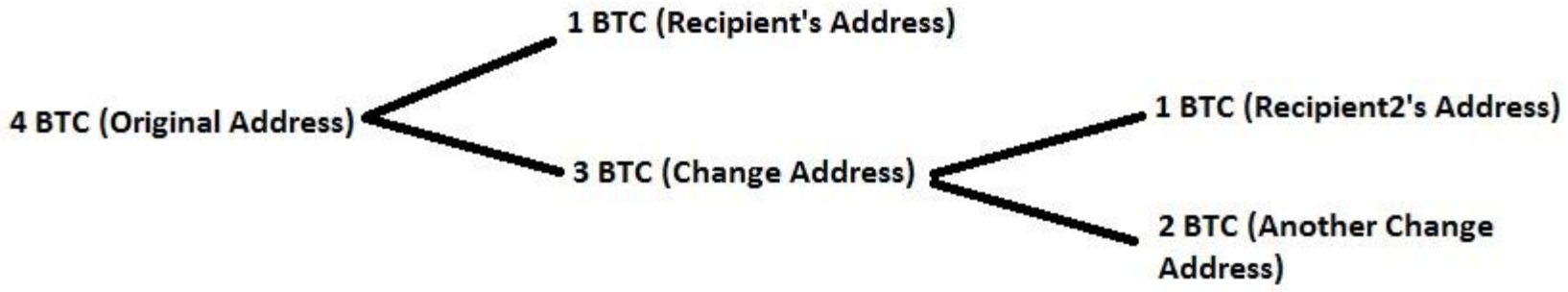
from owner to owner

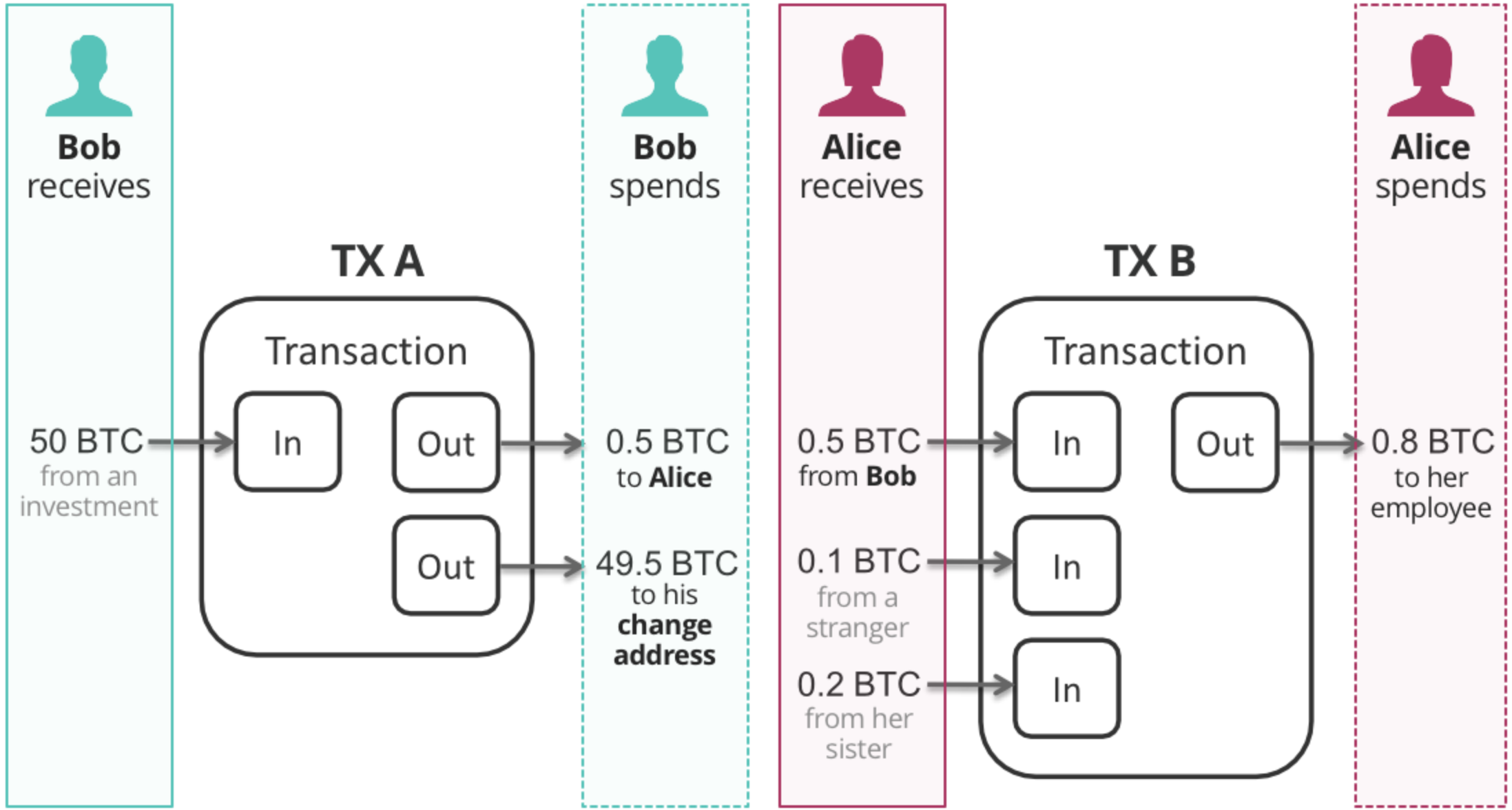
in a chain of transactions

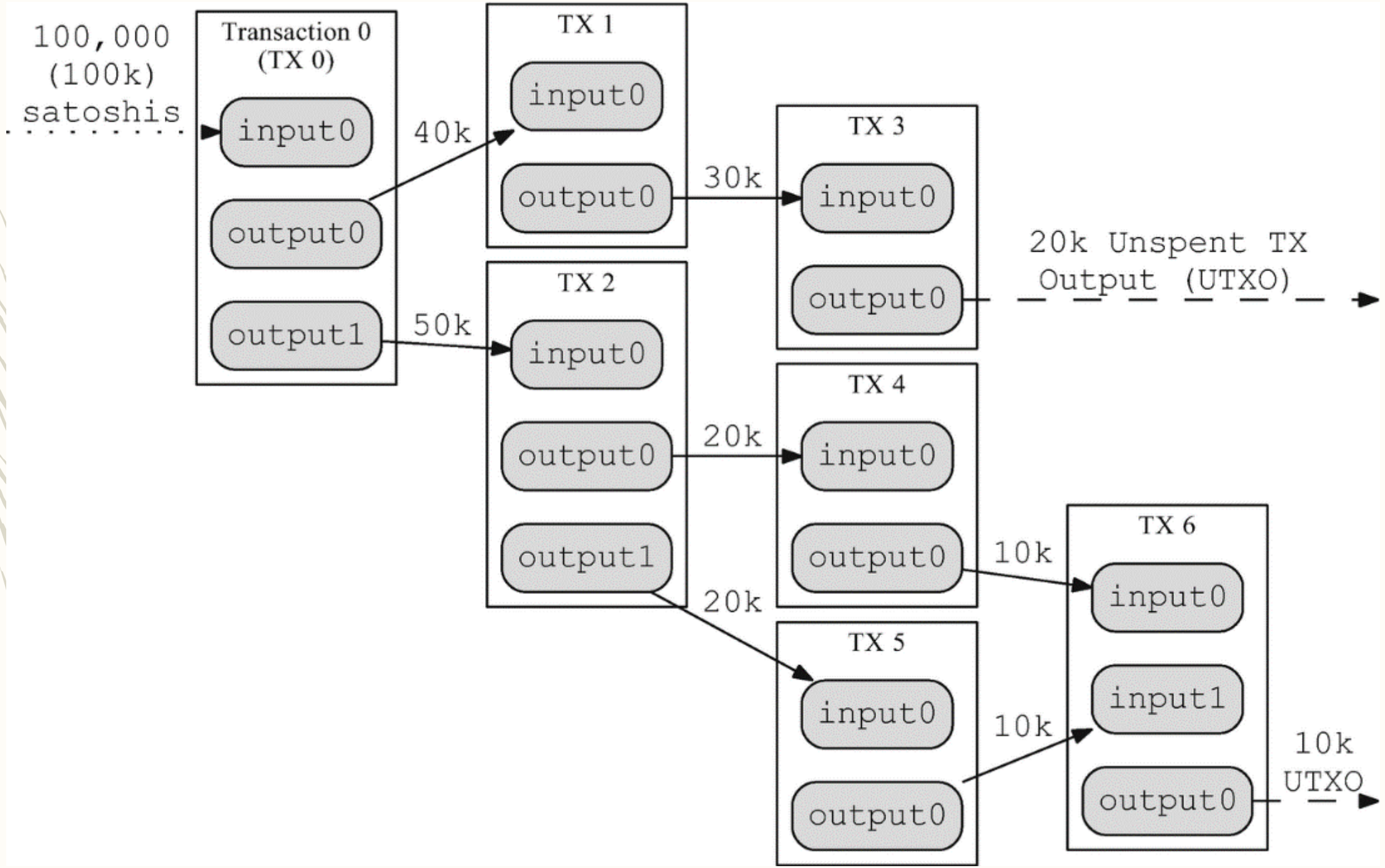
transaction is a node

consuming UTXO

and creating new UTXO




Bob
receives
50 BTC
from an
investment
TX A
Transaction
In
Out
Out
Bob
spends
0.5 BTC
to Alice
49.5 BTC
to his
change
address
Alice
receives
0.5 BTC
from Bob
0.1 BTC
from a
stranger
0.2 BTC
from her
sister
TX B
Transaction
In
In
In
Out
Alice
spends
0.8 BTC
to her
employee


100,000
(100k)
satoshis
Transaction 0
(TX 0)
input0
output0
output1
40k
50k
TX 1
input0
output0
TX 2
input0
output0
output1
30k
20k
20k
TX 3
input0
output0
TX 4
input0
output0
TX 5
input0
output0
20k Unspent TX
Output (UTXO)
10k
10k
TX 6
input0
input1
output0
10k
UTXO

Coinbase Transaction



Coinbase transaction

First transaction

in each block



Minting

creates brand-new

bitcoin payable



Miner's Reward

for successful mining

being the first among competitors



No UTXO consumption

Coinbase transaction

does not consume UTXO

Transaction Outputs



Every bitcoin transaction

creates outputs



Valid transactions

are recorded

on the bitcoin ledger



Create UTXO

create spendable chunks of bitcoin

recognized by the whole network



Can be spent

available for the owner

to spend in a future transactions



UTXO are tracked

by all full-node bitcoin clients

Transaction Output



Two parts

Transaction outputs consist of two parts



Amount

An amount of bitcoin denominated in *Satoshies*



Locking script

conditions to be able to spend the amount

Transaction Output

cryptographic puzzle

determines the conditions | required to spend the output

Other names

a *locking script* | a *witness script* | or a scriptPubKey

Transaction

Value

Cryptograhic puzzle

JSON encoding, the outputs are in an array (list) named **vout**

```
"vout": [
        {
            "value": 0.01500000,
            "scriptPubKey": "OP_DUP OP_HASH160 ab68025513c3dbd2f7b92a94e0581f5d50f654e7 OP_EQUALVERIFY OP_CHECKSIG"
        },
        {
            "value": 0.08450000,
            "scriptPubKey": "OP_DUP OP_HASH160 7f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a8 OP_EQUALVERIFY OP_CHECKSIG",
        }
]
```

Output-1

Output-2

Satoshis Not Bitcoins in Real

transaction contains **two outputs**

- a **value**
- a **cryptographic puzzle**.

the value

- is shown in **bitcoin**
- recorded as an **integer** denominated in **satoshis**.

the **cryptographic puzzle**

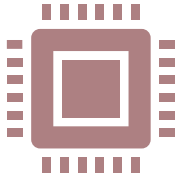
- sets the conditions for spending
- scriptPubKey
- human-readable representation of the script

Transaction serialization—outputs



Transaction Serialization

also known as a byte stream



Serialization

converting the data structure

to transmit one byte at a time



Usage

to transmit over the network

exchange between applications

storage in a file

Table 6-1. Transaction output serialization

Size	Field	Description
8 bytes (little-endian)	Amount	Bitcoin value in satoshis (10^{-8} bitcoin)
1–9 bytes (VarInt)	Locking-Script Size	Locking-Script length in bytes, to follow
Variable	Locking-Script	A script defining the conditions needed to spend the output

Transaction Storage



Not preferred

Storing transactions

internally as bytestreams



Reason

require complex parsing

to access a single field

Transaction Storage

Preferred storage	• store transactions internally • in data structures
Data structures	• usually object-oriented structures
Reason	• For convenience • and readability

Deserialization

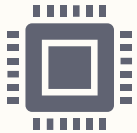


Deserialization

Parsing from

the byte-stream representation

to internal data structure



Serialization

Converting data structure

to Byte-stream



for

transmission over the network



bitcoin libraries

have built-in functions

for serialization and deserialization.

Example 6-1. Alice's transaction, serialized and presented in hexadecimal notation

0100000001186f9f998a5aa6f048e51dd8419a14d8a0f1a8a2836dd73
4d2804fe65fa35779000000008b483045022100884d142d86652a3f47
ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb02204b9f039
ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e3813
01410484ecc0d46f1918b30928fa0e4ed99f16a0fb4fde0735e7ade84
16ab9fe423cc5412336376789d172787ec3457eee41c04f4938de5cc1
7b4a10fa336a8d752adfffffffff02**60e316**0000000001976a914ab6
8025513c3dbd2f7b92a94e0581f5d50f654e788acd0ef800000000000
1976a9147f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a888ac 00000000

There are two outputs in the highlighted section, each serialized

- The value of 0.015 bitcoin is **1,500,000** satoshis.
- That's **16 e3 60** in hexadecimal.
- In the serialized transaction, the value **16 e3 60** is encoded in little-endian
- least-significant-byte-first byte order
- so it looks like **60 e3 16**
- The *scriptPubKey* length is **25** bytes, which is **19** in hexadecimal.

Transaction Inputs

Transaction inputs

- Identify UTXO
- to be consumed

Proof of Ownership

- provide proof of ownership
- through an unlocking script

Building a Transaction



To build a transaction,

a wallet selects from the UTXO it controls



Enough Value

UTXO must be with enough value

to make the requested payment.



Wallet creates

One input for each spending



Content

UTXO that will be consumed

an unlocking script

Components of an Input

Input content

UTXO reference

The 1st part

a pointer to an UTXO in Blockchain transction

and sequence number

Unlocking script

2nd part

constructed byt the wallet

To

satisfy the spending conditions

set in the UTXO

Components of an Input



unlocking script

proving ownership of the bitcoin



Content

Usually is a **digital signature** and public key.



Exceptions

not all unlocking scripts contain signatures

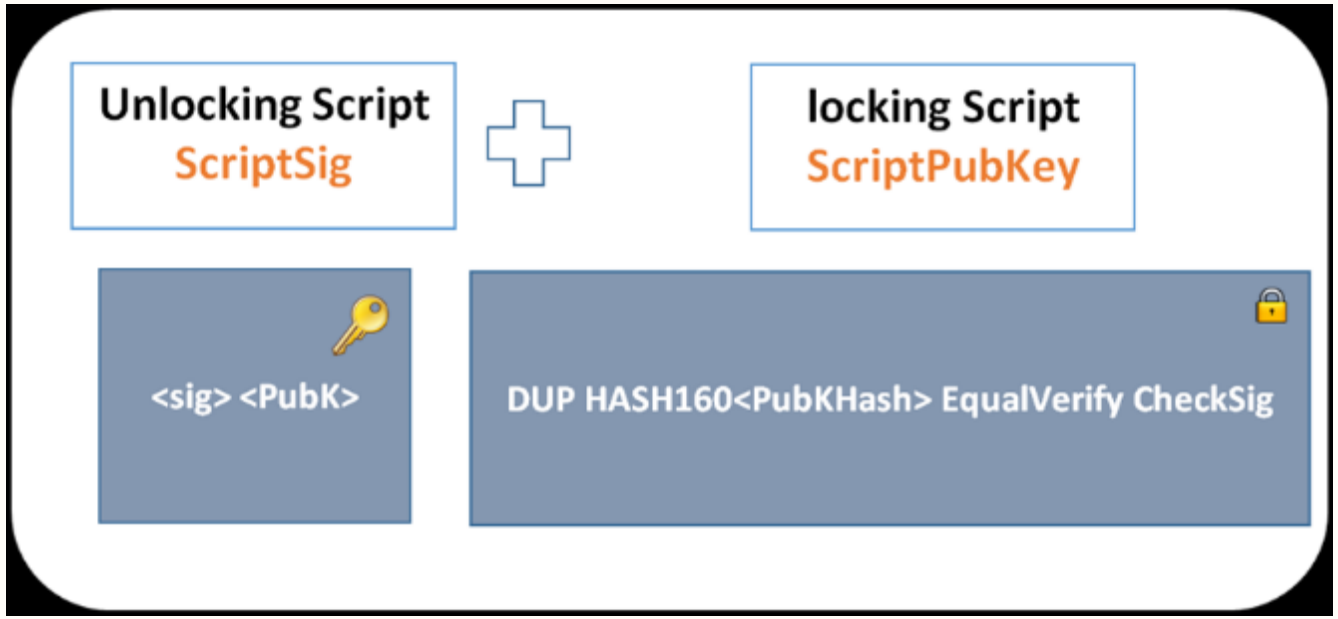
```
"vin": [
    {
        "txid": "7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18",
        "vout": 0,
        "scriptSig" :
"3045022100884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb02204
b9f039ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e3813[ALL]
0484ecc0d46f1918b30928fa0e4ed99f16a0fb4fde0735e7ade8416ab9fe423cc5412336376789d1
72787ec3457eee41c04f4938de5cc17b4a10fa336a8d752adf",
        "sequence": 4294967295
```


- There is only one input in the list
  - because one UTXO contained sufficient value to make this payment.
- The input contains four elements:

1. **A transaction ID**, referencing the transaction that contains the UTXO being spent
2. **An output index** (vout), identifying **which UTXO** from that transaction is referenced (first one is zero)
3. **A scriptSig**, which satisfies the conditions placed on the UTXO, unlocking it for spending
4. **A sequence number** (to be discussed later)

In Alice's transaction, the input points to the transaction ID:
7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18
and output index 0 (i.e., the first UTXO created by that transaction).

Unlocking Script





The unlocking script

is constructed

by sender’s wallet

Unlocking Script



The unlocking script

Construction steps



1

retrieve the referenced UTXO



2

examine its locking script



3

build the unlocking script to satisfy it

What is Known about UTXO?



Transaction input

is just a reference to the transaction

containing UTXO



Retrieve

Transaction of UTXO

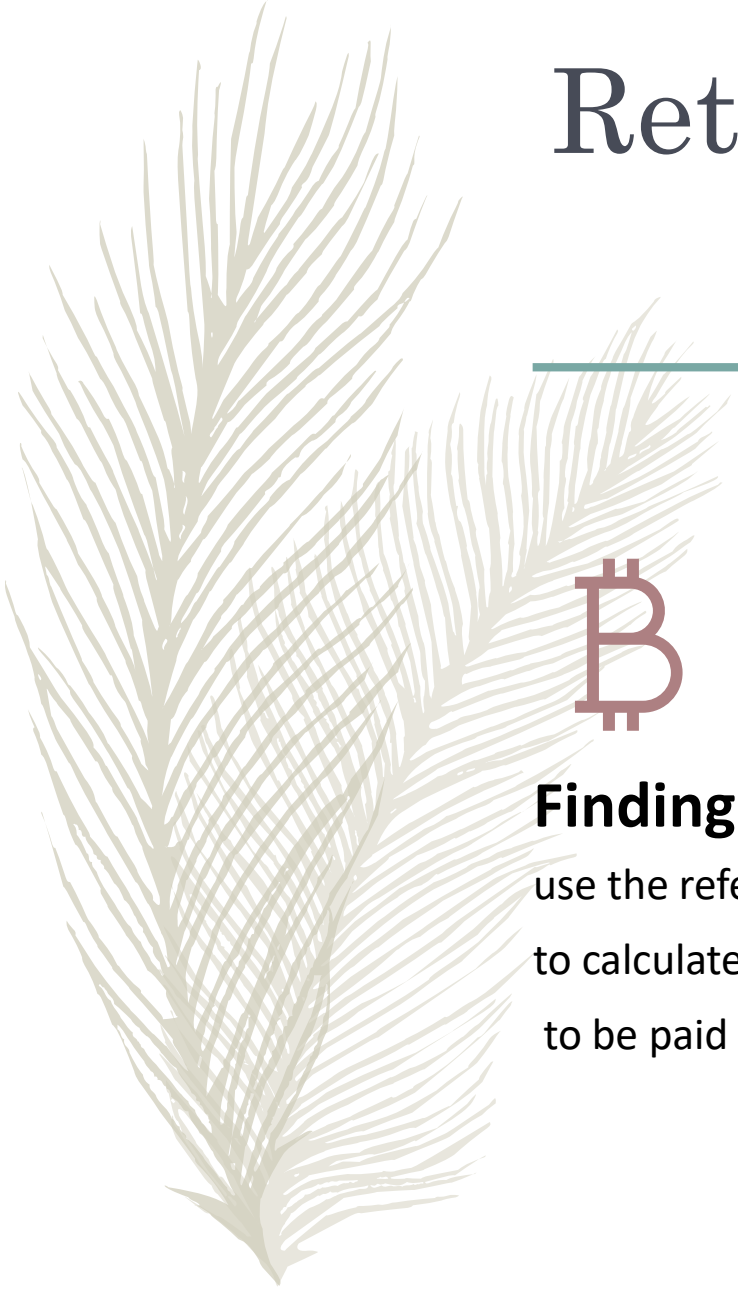


Details

the amount in Satoshies

the locking script

Retrieving UTXO



Finding the fee

use the referenced UTXO

to calculate the fees

to be paid in the transaction

Reason

the value of the input

is not explicitly stated

Validation process

every validating node

will also to retrieve the UTXO

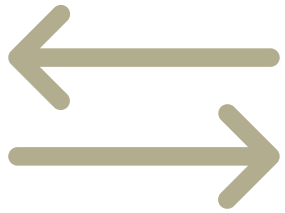
to validate the transaction

Incompleteness of Transactions



Transactions on their own seem incomplete

because they lack context.



They reference UTXO in their inputs

but without retrieving that UTXO
we cannot know the value of the inputs
or their locking conditions.

Simple Operations to Multiple Steps



For example,

to calculate the amount paid in fees,

you must know the sum of the values of inputs and outputs.



But without retrieving the UTXO referenced in the inputs,

you do not know their value.



So a seemingly simple operation

like counting fees in a single transaction

in fact involves multiple steps

and data from multiple transactions.

Retrieve referenced UTXO

use the same sequence of commands with Bitcoin Core

to get the UTXO referenced

in the preceding input

```
"vout": [
        {
                "value": 0.10000000,
                "scriptPubKey": "OP_DUP OP_HASH160
                7f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a8 OP_EQUALVERIFY OP_CHECKSIG"
        }
]
```

This UTXO

- has a value of 0.1 BTC
- has a locking script

script PubKey

- contains “OP_DUP OP_HASH160 7f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a8 OP_EQUALVERIFY OP_CHECKSIG”.



To fully understand Alice's transaction we had to retrieve the previous transaction(s) referenced as inputs. A function that retrieves previous transactions and unspent transaction outputs is very common and exists in almost every bitcoin library and API.

Transaction serialization—inputs

When transactions are serialized for transmission on the network,

- their inputs are encoded
- into a byte stream

Table 6-2. Transaction input serialization

Size	Field	Description
32 bytes	Transaction Hash	Pointer to the transaction containing the UTXO to be spent
4 bytes	Output Index	The index number of the UTXO to be spent; first one is 0
1–9 bytes (VarInt)	Unlocking-Script Size	Unlocking-Script length in bytes, to follow
Variable	Unlocking-Script	A script that fulfills the conditions of the UTXO locking script
4 bytes	Sequence Number	Used for locktime or disabled (0xFFFFFFFF)

Transaction Fees



transaction fees

Optional

Rationale

compensate the bitcoin miners

for securing the network

Additional security

makes it economically infeasible for attackers

to flood the network with transactions

Fee inclusion

Most wallets calculate

and include transaction fees automatically.

Programmatic & Command line

must manually be accounted for

and include transactions

Transaction Fees



Prevents abuse

serve as an incentive

by imposing a small cost on every transaction



Miner collects

fees are collected by the miner



Fees on Kilobytes

Fees are based on the **size** of the transaction in kilobytes

not the value of the transaction in bitcoin.



Market influenced

Overall, transaction fees

based on market forces within the bitcoin network.

Transaction Fees



Miners prioritize transactions

based on many different criteria

including fees,

and might even process transactions for free

under certain circumstances.



Priority of Transaction

Fees affect the processing priority



Fast process

a transaction with sufficient fees

is likely to be included in the next block mined,



Delayed process

a transaction with insufficient or no fees

might be delayed

or not processed at all

Evolution of Fee Policy



First approach: Fixed fees

, transaction fees were fixed

constant across the network.



Gradually relaxed

influenced by market forces

network capacity

transaction volume



Radical change from 2016

capacity limits created competition between transactions

resulting in higher fees

making free transactions a thing of the past



Not minded transactions

Zero fee or very low fee transactions

rarely get mined

even be propagated across the network.

Bitcoin Core Fee Relay Policies



minrelaytxfee

In Bitcoin Core, fee relay policies

are set by the *minrelaytxfee* option.



current default

minrelaytxfee is

0.00001 bitcoin/KB

a hundredth of a millibitcoin per kilobyte.



Free

transactions with a fee less than 0.0001 bitcoin

are treated as free

are only relayed if there is space in the mempool

otherwise, they are dropped.



Override default

Bitcoin nodes can override the default fee relay policy

by adjusting the value of *minrelaytxfee*

Dynamic Fees



Any bitcoin service that creates transactions,

including wallets, exchanges, retail applications, etc.,

must implement dynamic fees



Dynamic fees can be implemented

through a third-party fee estimation service

or with a built-in fee estimation algorithm.



If you're unsure

begin with a third-party service



as you gain experience

design and implement your own algorithm

to remove the third-party dependency.

Fee Estimation Algorithms



calculate the appropriate fee

based on capacity and the fees offered by “competing” transactions



These algorithms range

from simplistic

- average
- median fee in the last block

to sophisticated

- statistical analysis)



They estimate

the necessary fee (in satoshis per byte) that will give a transaction a high probability of being selected and included within a certain number of blocks.

Fee Estimation Algorithms



Most services offer users

the option of choosing

high, medium, or low priority fees.



High priority means

users pay higher fees but the transaction is likely

to be included in the next block.



Medium and low priority means

users pay lower transaction fees

but the transactions may take much longer to confirm.

https://bitcoinfees.earn.com/

Adding Fees to Transactions



The data structure of transactions

does not have a field for fees.



fees are implied

as the difference between the sum of inputs

and the sum of outputs.



Any excess amount

that remains after all outputs

have been deducted



from all inputs is the fee

that is collected by the miners:

Fees = Sum(Inputs) – Sum(Outputs)

Adding Fees to Transactions



if you consume a 20-bitcoin UTXO

to make a 1-bitcoin payment,



You must include a 19-bitcoin change output

back to your wallet.



Otherwise, the 19-bitcoin “leftover”

will be counted as a transaction fee and will be collected by the miner



Transaction Script & Script Language

- Turing Incompleteness
- Stateless Verification
- Script Construction
- Pay-to-Public-Key-Hash

Transaction Scripts and Script Language



Script

The bitcoin transaction script language,

a Forth-like reverse-polish notation stack-based execution language.



very simple language

designed to be limited in scope



Wide range of Hardware

executable on a range of hardware,

perhaps as simple as an embedded device.

requires minimal processing



security feature

cannot do many of the fancy things

modern programming languages can do.

Script Types



Today, most transactions on Bitcoin

have the form “Payment to Bob’s bitcoin address”

are based on a script called a Pay-to-Public-Key-Hash script.



However, bitcoin transactions

are not limited to the “Payment to Bob’s bitcoin address” script.



locking scripts can be written

to express a vast variety of complex conditions.

Programmable Money



Bitcoin transaction validation

is not based on a static pattern,



achieved through

the execution of a scripting language.



This language allows

for a nearly infinite variety of conditions

to be expressed.



This is how bitcoin gets

the power of “programmable money.”

Turing Incompleteness

The bitcoin transaction script language

contains many operators,

limited in one important way

there are no loops

or complex flow control capabilities

other than conditional flow control.

This ensures that

the language is **not** *Turing Complete*

meaning that

scripts have **limited complexity**

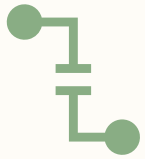
and predictable execution times.

Turing Incompleteness



Limitation

This is a limited language



To prevent vulnerability

against

the transaction validation mechanism

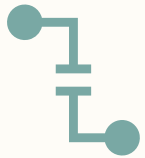


***Script* cannot be used**

to create an infinite loop

or other form of “logic bomb”

Turing Incompleteness



DAD Risk

DAD denial-of-service attack

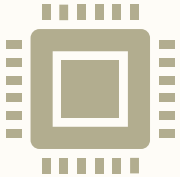
against the bitcoin network



Method

infinite loop

could be embedded in a transaction



Impact on Network

every transaction is **validated**

by every **full node**

on the bitcoin network

Stateless Verification



Script is stateless

there is no state prior to execution of the script

or state saved after execution of the script.



all the information needed to execute a script

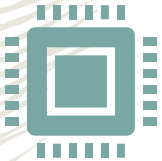
is contained within the script



A script will predictably execute

the same way on any system.

Stateless Verification



If your system verifies a script,

you can be sure that every other system in the bitcoin network

will also verify the script



a valid transaction

is valid for everyone

and everyone knows this



Predictability is essential

an essential benefit of the bitcoin system.

Script Construction (Lock + Unlock)



Validation

Bitcoin’s transaction validation engine



two types of scripts

to validate transactions

locking script

unlocking script



locking script

spending conditions

placed on an output



unlocking script

allows the output to be spent

Script Construction (Lock + Unlock)



locking script

specifies the conditions

to spend the output in the future



unlocking script

“solves”: satisfies the conditions

placed on an output

Locking Script: *scriptPubKey*



Historical name

scriptPubKey

contains

- **public** key
- or **bitcoin address**



bitcoin address

hash of a public key



Other names

witness script

cryptographic puzzle

Unlocking Script: *scriptSig*



Unlocking script

Historical name: *scriptSig*

exists in every transaction input



scriptSig = Witness

Can also be referred to

as a *witness*



Digital Signature

contain a digital signature

produced from the private key

Execution on Bitcoin Nodes



Every Bitcoin node

will validate

transactions



Validation

executing the locking

and unlocking scripts together



UTXO

unlocking script

refers to a previously existing UTXO

Validation



unlocking script

retrieve the UTXO

copy the locking script

from that UTXO



Execute

unlocking and locking script

in sequence



The input is valid

if the unlocking script

satisfies the locking script conditions

Verification

Locking script (Previous UTXO-Alice)	DUP HASH160 <PubKHash> EQUALVERIFY CHCKSIG
Unlocking script (INP-Bob)	<sig> <pubK>
Verification script	Unlocking script Locking Script
	<sig> <pubK> DUP HASH160 <PubKHash> EQUALVERIFY CHCKSIG

payment to a public key hash example	combined script
•unlocking script •locking script	•resulting from the concatenation of •The unlocking and locking scripts •prior to script validation

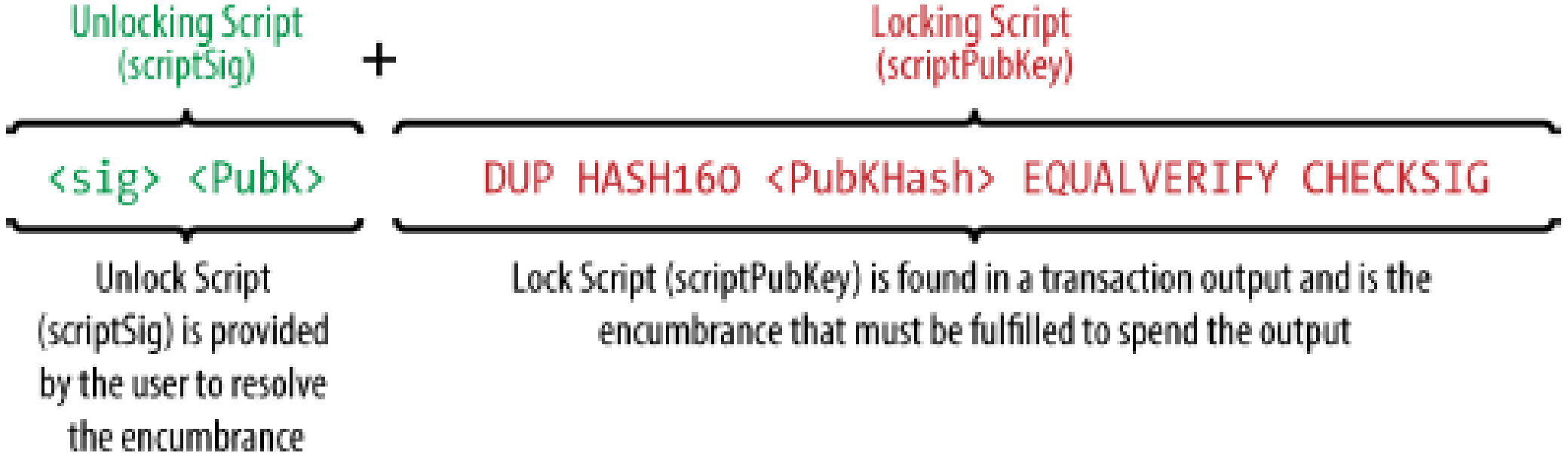


Figure 6-3. Combining scriptSig and scriptPubKey to evaluate a transaction script

Protection of UTXO



Protected

only valid transaction can spend

unaffected by failed attempts



Permenant

permanently recorded

in the blockchain

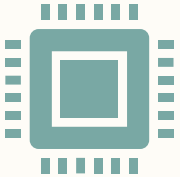


Valid transaction

correctly satisfies output conditions

removes from UTXO SET

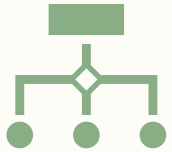
The Script Execution Stack



Stack-based

Bitcoin's scripting language

is a stack-based language



Stack-based

Script is processed

using a stack



Processing

executes the script

by processing each item

from left to right.

The Script Execution Stack



Push Numbers

data **constants**

are pushed onto the stack.



Pop for Operators

pop **numbers** from the stack

act on them



Push the results

push a **result**

onto the stack

The Script Operator OP_ADD



OP_ADD



Pop two items

will pop two items from the stack



Add

add them



Push the reuls

push the resulting sum onto the stack

Conditional Operators



Conditional operators

evaluate a condition

produces a Boolean result



Boolean Results

TRUE

FALSE



TRUE

represented by

the number **1**



FALSE

represented by

the number **zero**

Conditional Operator: OP_EQUAL



pops

two items

from the stack



Push

TRUE: they are equal

FALSE: they are not equal



Remaining Stack

TRUE or

any other result

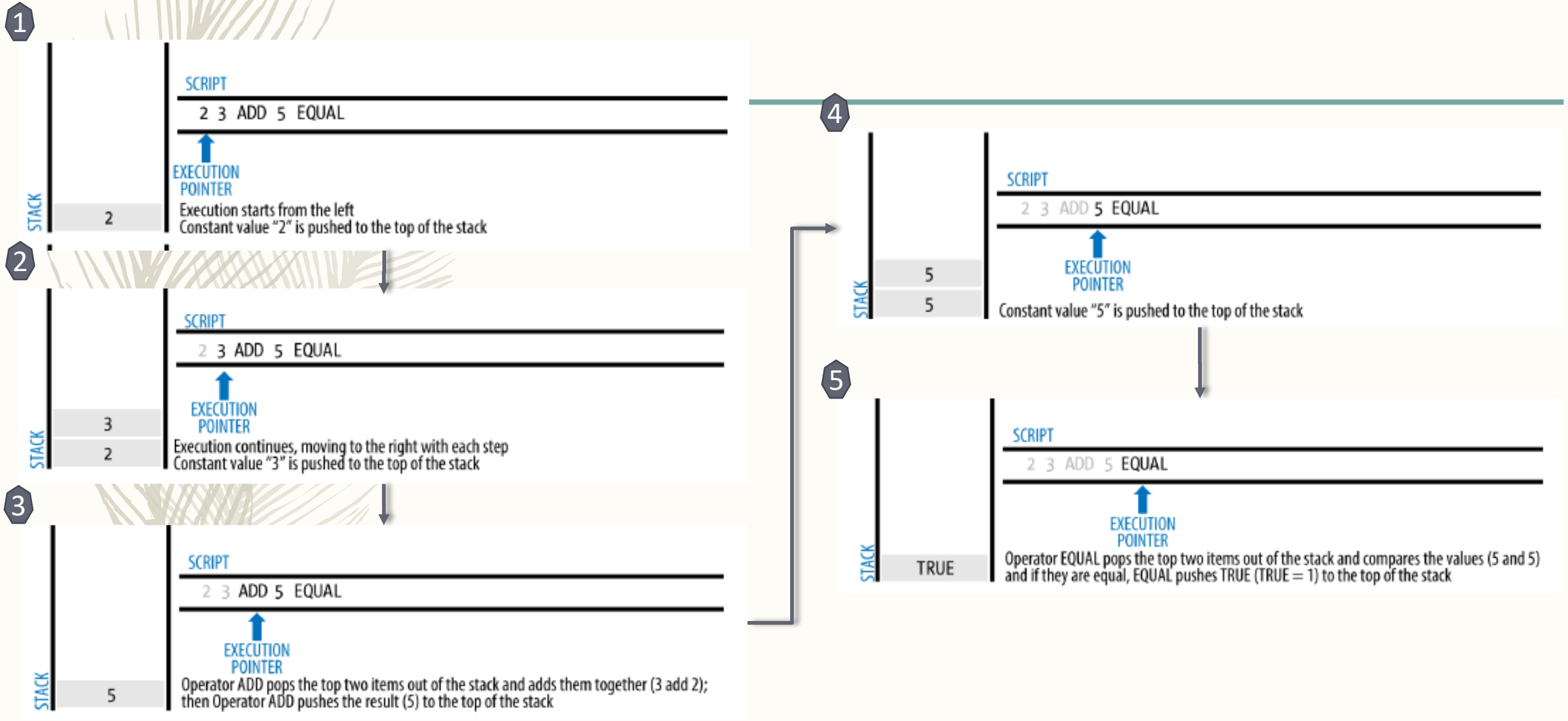


Valid

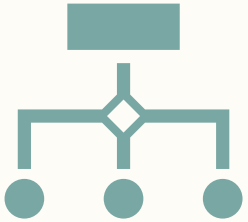
if the result is TRUE

signifies a valid transaction

A Simple Script



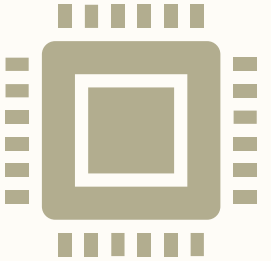
Original Execution of Scripts



Original approach

for execution

unlocking and locking scripts



Execution

scripts are concatenated

and executed in sequence

2010 Separation of Execution



Execution separated

In 2010 the scripts are executed separately



Security

For security reasons due to a vulnerability



malformed unlocking script

to push data onto the stack and corrupt the locking script

Separate Execution of Scripts



Start with unlocking script

the unlocking script

is executed first



If no error encountered

the main stack is copied

the locking script is executed



No error means

no “dangling” operators

left over

Separate Execution of Scripts



Success of unlocking script

result is “TRUE”

the unlocking script has succeeded



Outcome

locking script conditions

are resolved



TRUE means

a valid spend

the input is a valid authorization

to spend the UTXO

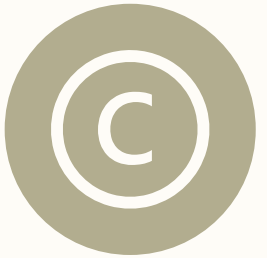
Separate Execution of Scripts



invalid

If any result

other than “TRUE”



May remain

after execution of

the combined script

Pay-to-Public-Key-Hash (P2PKH)



Pay-to-Public-Key-Hash

spend outputs locked

with a public key



Common in Bitcoin

majority of transactions

processed on the bitcoin network



Contains

a locking script

locks the output

to a public key hash

Pay-to-Public-Key-Hash (P2PKH)



Public key hash

known as a bitcoin address

generated by hashing public key



Spending

unlocking P2PKH script output

by presenting a public key

and a digital signature



digital signature

produced from the private key

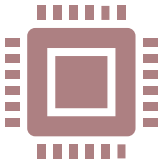
of the public key

Alice's Payment: Locking Script



Alice made a payment of 0.015 bitcoin

to the cafe's bitcoin address.



That transaction output

would have a locking script



Locking script

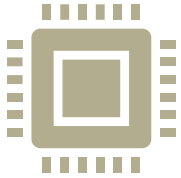
OP_DUP OP_HASH160 <Cafe Public Key Hash> OP_EQUALVERIFY OP_CHECKSIG

Alice's Payment: Locking Script



Locking script

OP_DUP OP_HASH160 <Cafe Public Key Hash> OP_EQUALVERIFY OP_CHECKSIG



The Cafe Public Key Hash

is bitcoin address of the cafe

without the Base58Check encoding.



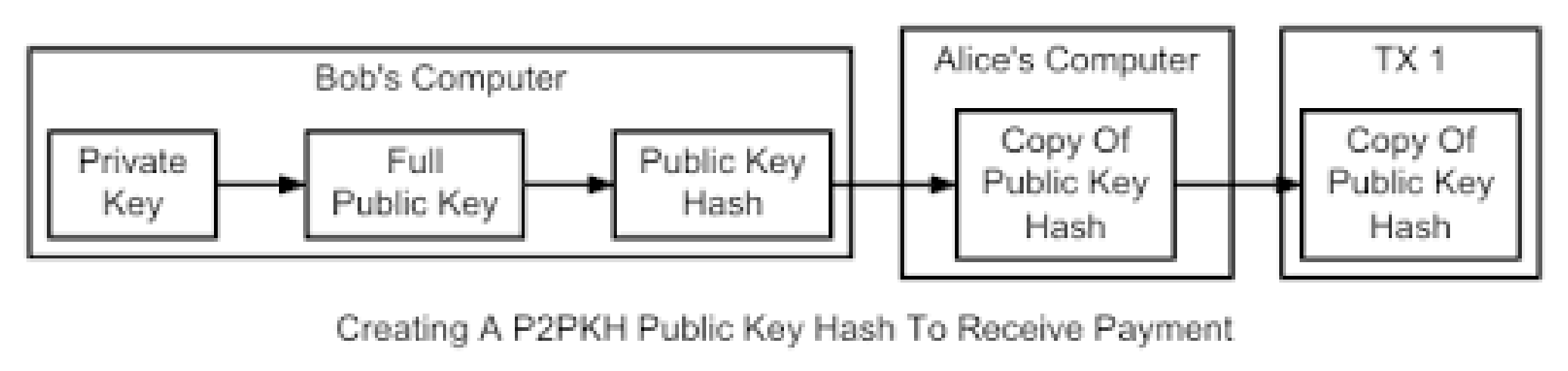
hexadecimal encoding

Most applications show the *public key hash*

in hexadecimal encoding

not Base58Check format that begins with a "1."

Alice’s Payment: Locking Script



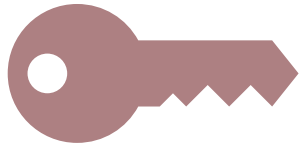
Creating A P2PKH Public Key Hash To Receive Payment

Bob's Spending: Unlocking Script



Locking script

OP_DUP OP_HASH160 <Cafe Public Key Hash>
OP_EQUALVERIFY OP_CHECKSIG

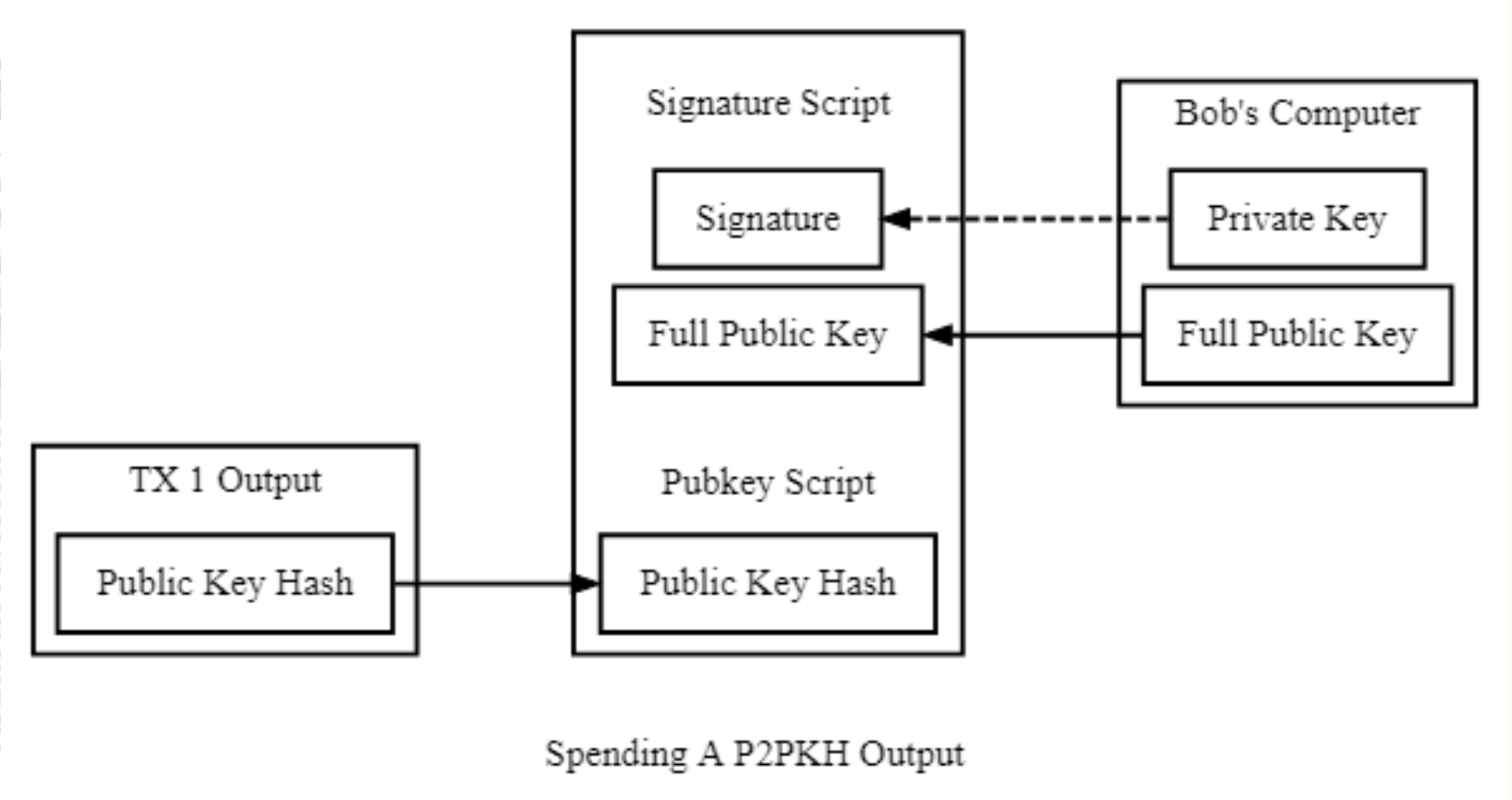


To be specified

need an unlocking script of the form

<Cafe Signature> <Cafe Public Key>

Alice’s Payment, Bob’s Spending



Spending A P2PKH Output

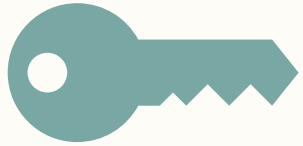
Bob's Spending: Validation



Combination

The two scripts together

would form the combined validation script



validation script

<Cafe Signature> <Cafe Public Key> OP_DUP OP_HASH160

<Cafe Public Key Hash> OP_EQUALVERIFY OP_CHECKSIG

Bob's Spending: Validation



Evaluate to TRUE

When executed combined script

will evaluate to TRUE



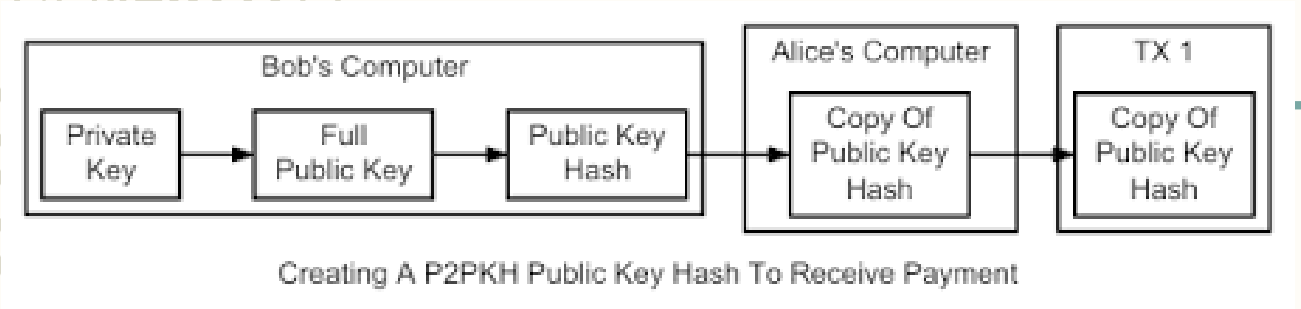
Condition

if, and only if

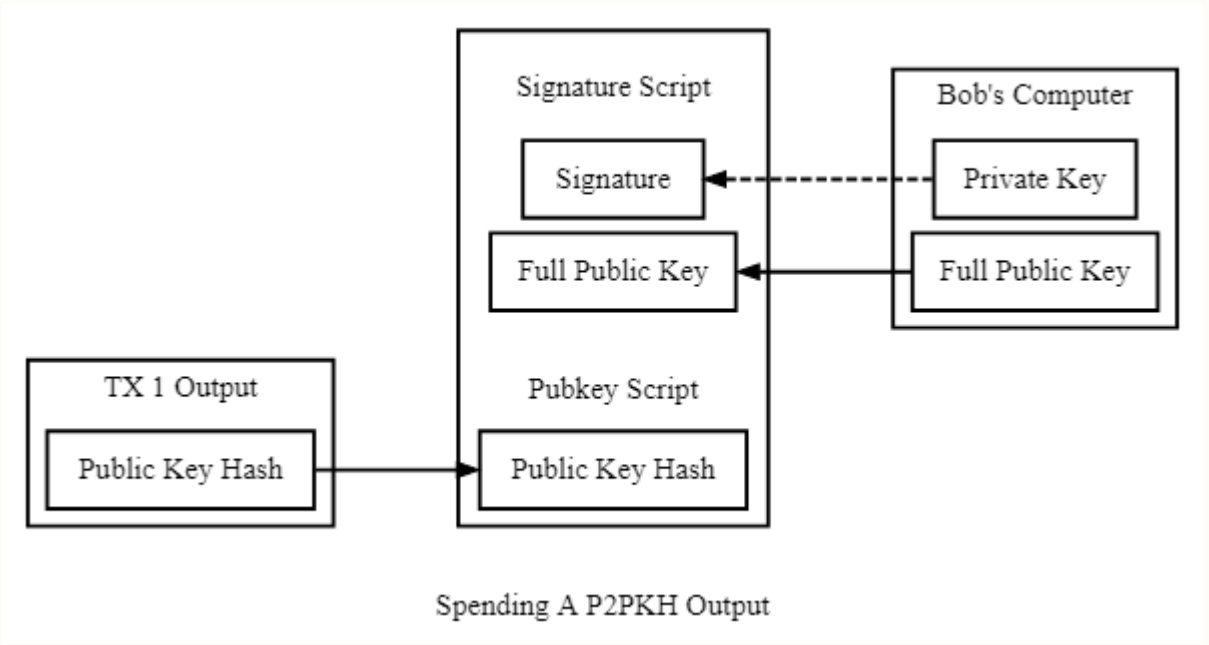
the unlocking script matches

the conditions set by the locking script. I

Alice's Payment, Bob's Spending



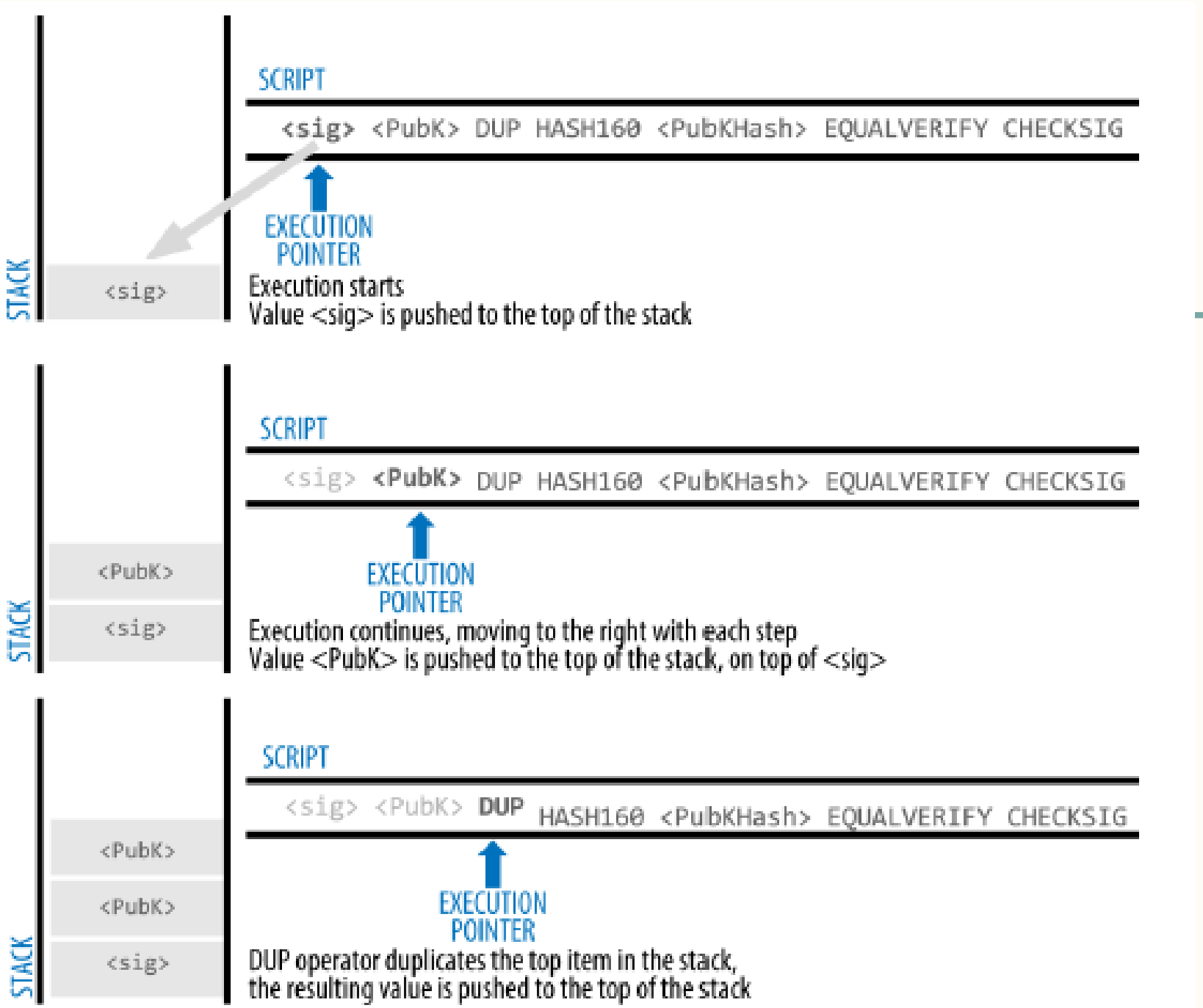
Creating A P2PKH Public Key Hash To Receive Payment

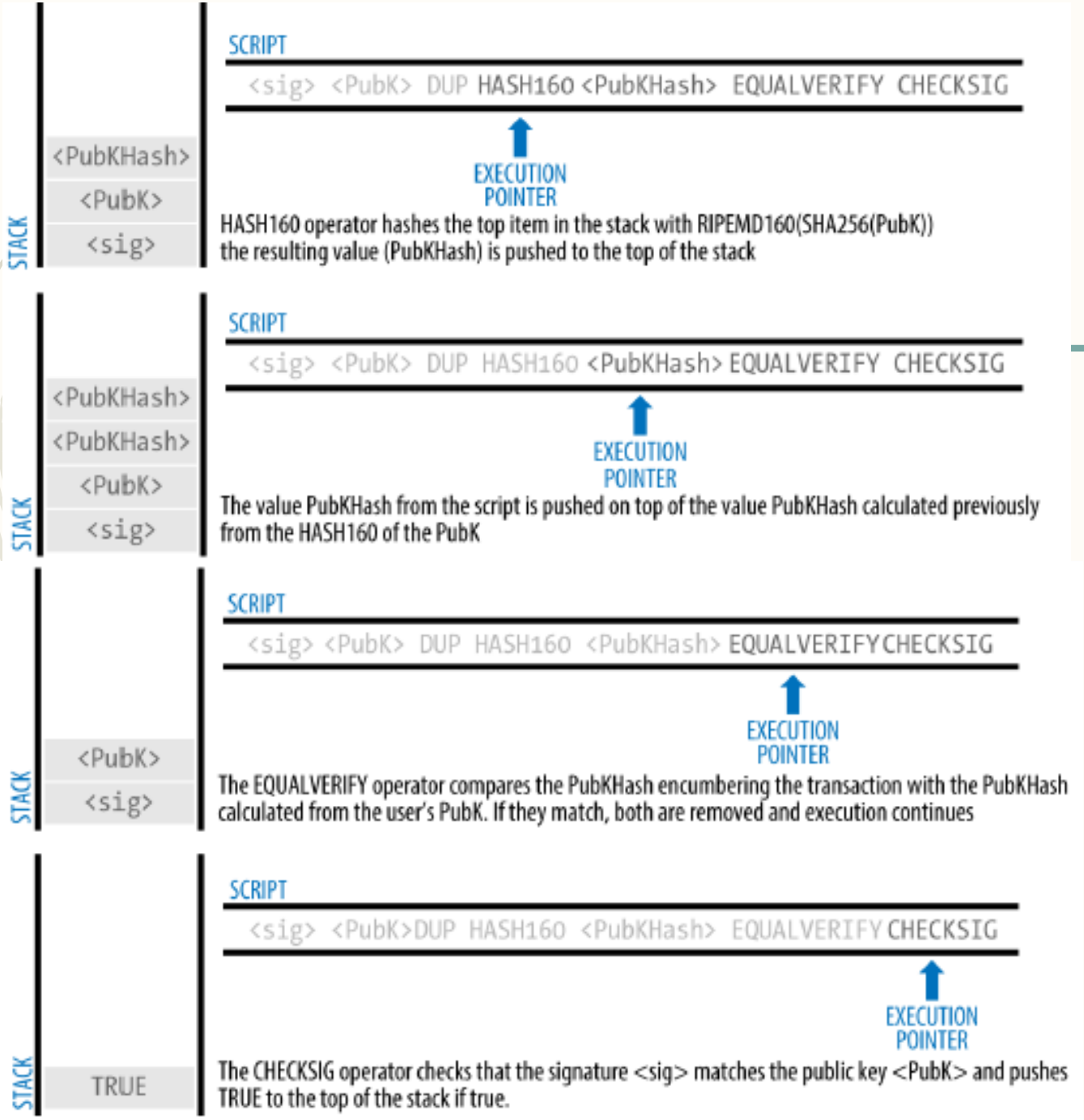


Spending A P2PKH Output

Execution of Locking
& Unlocking Scripts

Steps of Validation


SCRIPT
<sig> <PubK> DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
STACK
<sig>
Execution starts
Value <sig> is pushed to the top of the stack
SCRIPT
<sig> <PubK> DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
STACK
<PubK>
<sig>
Execution continues, moving to the right with each step
Value <PubK> is pushed to the top of the stack, on top of <sig>
SCRIPT
<sig> <PubK> DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
STACK
<PubK>
<PubK>
<sig>
DUP operator duplicates the top item in the stack,
the resulting value is pushed to the top of the stack


STACK
<PubKHash>
<PubK>
<sig>
SCRIPT
<sig> <PubK> DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
HASH160 operator hashes the top item in the stack with RIPEMD160(SHA256(PubK)) the resulting value (PubKHash) is pushed to the top of the stack
STACK
<PubKHash>
<PubKHash>
<PubK>
<sig>
SCRIPT
<sig> <PubK> DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
The value PubKHash from the script is pushed on top of the value PubKHash calculated previously from the HASH160 of the PubK
STACK
<PubK>
<sig>
SCRIPT
<sig> <PubK> DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
The EQUALVERIFY operator compares the PubKHash encumbering the transaction with the PubKHash calculated from the user's PubK. If they match, both are removed and execution continues
STACK
TRUE
SCRIPT
<sig> <PubK>DUP HASH160 <PubKHash> EQUALVERIFY CHECKSIG
EXECUTION POINTER
The CHECKSIG operator checks that the signature <sig> matches the public key <PubK> and pushes TRUE to the top of the stack if true.

Digital Signatures



How Digital Signatures Work



Verifying the Signature



Signature Hash Types



ECDSA Math



The Importance of Randomness

Digital Signatures (ECDSA)



ECDSA

Elliptic Curve Digital Signature Algorithm

based on elliptic curve private/public key pairs



Bitcoin's Algorithm

The digital signature algorithm

used in bitcoin



script functions

OP_CHECKSIG

OP_CHECKSIGVERIFY

OP_CHECKMULTISIG

OP_CHECKMULTISIGVERIFY

Digital Signature Usage



Owner

has *authorization*

in spending of funds



Proof of Ownership

the digital signature

implies the ownership



Digital signature

proves the ownership

to the private key

Digital Signature Usage



Transaction Unmodifiability

the signature proves
have not and *cannot be modified*



Validation

Once signed by anyone
after it has been signed



Full or Partial

transaction fully
or its specific parts

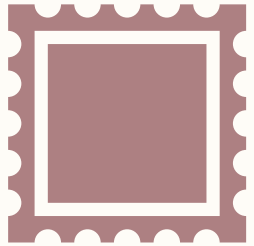
How Digital Signatures Work



Mathematical scheme

A digital signature is

a *mathematical scheme*



consists of two algorithms

signature generation

signature verification

How Digital Signatures Work

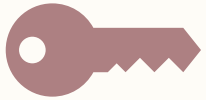


Signature generation

signature is generated

using a signing key

and a message



the signing key

the private key



message

the transaction

How Digital Signatures Work

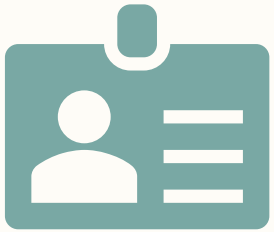


Signature verification

an algorithm

allows anyone

to verify the signature



given

the message

and a public key

Creating a Digital Signature



Formula

$Sig = F_{sig}\ (F_{hash}\ (m)\ ,\ dA)$



Sig

the signature

Sig = (R, S)



dA

the signing private key



m

the transaction

or parts of it



F_{hash}

the hashing function



F_{sig}

the signing algorithm

Serialization of Signatures (DER)



Sig = (R, S)

two values R and S

have been calculated



Serialization

R & S are

serialized into a bytestream



DER

Distinguished Encoding Rules

international standard encoding scheme

Alice’s case



transaction input

unlocking script contains

DER-encoded signature



Wallet generation

Generated from Alice’s wallet

Signature having R & S



R

3045022100884d142d866
52a3f47ba4746ec719bbfb
d040a570b1deccbb6498c
75c4ae24cb02204b



S

9f039ff08df09cbe9f6adda
c960298cad530a863ea8f5
3982c09db8f6e381301

Serialization of Signatures (DER)



That serialized signature

produced by Alice’s wallet



Signature proves

Alice owns the private key authorized to spend that output.



9 Elements

The serialization format consists of nine elements

The Serialization Format

3045**02**21**00884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb**
02**20**4b9f039ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e381301

0x30—indicating the start of a DER sequence

0x45—the length of the sequence (69 bytes)

0x02—an integer value follows

0x21—the length of the integer (33 bytes)

R—00884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb

0x02—another integer follows

0x20—the length of the integer (32 bytes)

S—4b9f039ff08df09cbe9f6addac960298cad530a863ea8f53982c09db8f6e3813

A suffix (0x01) indicating the type of hash used (SIGHASH_ALL)

Verifying the Signature



Verification

Certain conditions must hold



Meaning of verification



“Only the owner

of the private key that
generated this public key



could have produced

this signature
on this transaction.

Verifying the Signature



Public & Private keys

Public key is generated

by the owner of private key



Private Key & Signature

the owner of the private key

can generate the digital signature



Transaction

The owner of private key

Can generate

The signature usint the transaction



Verification Fields

The signature: R & S

the serialized transaction

the public key

Verifying the Signature



The signature verification algorithm

takes the message



Hash

a hash of the transaction or parts of it



Public key

the signer's public key



Signature

R and S values



returns TRUE

if the signature is valid for this message and public key.



Bitcoin Addresses, Balances, & Other Abstractions

- Bitcoin Addresses
- Balances
- Other Abstractions

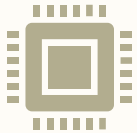
Constructing the “Final Balance”



The blockchain explorer

keeps a separate database

UTXO set



Monitoring

the blockchain explorer

monitor the bitcoin network

for arrival of new transactions



Continues operation

add newly created UTXO

remove spent UTXO



Real time processing

as they appear in

unconfirmed transactions

Constructing the “Final Balance”



Final balance

a complicated process



depends on

keeping track of

transactions as they propagate



plus

maintaining consensus

with the bitcoin network

Constructing the “Final Balance”



Going out of sync

Sometimes the blockchain explorer goes out of sync



Causes

its perspective of the UTXO set is incomplete or incorrect.

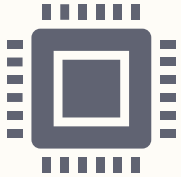
Constructing the “Final Balance”



Final balance

the number

shown to the user



produced by

the blockchain explorer



Operation

Sum up the value of

all unspent outputs

Constructing the “Final Balance”



Bitcoin address

referencing public key hash

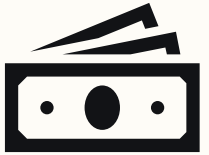
in UTXO set



Finding the balance

requires to search

indexes and transactions



of transactions

dozens

hundreds

hundreds of thousands

Transaction View information about a bitcoin transaction

0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

Retrieved from previous UTXO

Alice's Bitcoin Address

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK (0.1 BTC - Output)

Sender

Receiver

Bob's Bitcoin Address

1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA - (Unspent) 0.015 BTC

Paid

Alice's Bitcoin Address

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK - (Unspent) 0.0845 BTC

Change

97 Confirmations | 0.0995 BTC

Summary	
Size	258 (bytes)
Received Time	2013-12-27 23:03:05
Included In Blocks	277316 (2013-12-27 23:11:54 +9 minutes)

Inputs and Outputs	
Total Input	0.1 BTC
Total Output	0.0995 BTC
Fees	0.0005 BTC
Estimated BTC Transacted	0.015 BTC

Fee

Bob's Balance

Bitcoin Address

Addresses are identifiers which you use to send bitcoins to another person.

Summary	
Address	1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA
Hash 160	ab68025513c3dbd2f7b92a94e0581f5d50f654e7
Tools	Taint Analysis - Related Tags - Unspent Outputs

Transactions		
No. Transactions	25	
Total Received	0.17579525 BTC	
Final Balance	0.17579525 BTC	

Discussions

Your opinion
What do you think on this chapter?

Challenging
What is the most interesting point?

Clarification
What are the unclarified points?

Critical issue
Select and explain a critical issu in this chapter

